

Registro de decisiones de arquitectura de UdeSA-X

Cada archivo registra una decisión con su contexto, la decisión tomada, las alternativas y sus consecuencias, en el formato de Michael Nygard.

Estas decisiones ya estaban tomadas e implementadas. Se registraron el 2 de octubre de 2026 a partir del código, las guías de cada repositorio y los pull requests. La fecha de cada una es la del primer cambio que la puso en práctica.

Número	Decisión	Fecha	Estado
ADR-0001	Microservicios por contexto acotado, no por épica	2026-08-11	Aceptada
ADR-0002	Una base de datos por servicio, sin claves foráneas entre servicios	2026-08-19	Aceptada
ADR-0003	PostgreSQL como fuente de verdad y Redis para datos que vencen	2026-08-13	Aceptada
ADR-0004	Python con FastAPI y SQLAlchemy 2.0 sincrónico	2026-08-11	Aceptada
ADR-0005	admin-service escrito en Go	2026-09-17	Aceptada
ADR-0006	Sesiones con JWT firmado y revocable	2026-08-16	Aceptada
ADR-0007	API gateway propio con verificación central del token	2026-09-14	Aceptada
ADR-0008	La base garantiza unicidad, idempotencia y contadores	2026-08-13	Aceptada
ADR-0009	Paginación por cursor en las listas	2026-08-31	Aceptada
ADR-0010	Comunicación sincrónica entre servicios, en lote y con falla cerrada	2026-09-10	Aceptada
ADR-0011	Baja lógica de usuarios y posts	2026-08-18	Aceptada
ADR-0012	Despliegue en el cluster EKS compartido de la cátedra	2026-10-02	Aceptada
ADR-0013	Testing con dependencias inyectadas, base real y mínimo de cobertura	2026-09-04	Aceptada
ADR-0014	App mobile con Expo, TanStack Query y Zustand	2026-08-25	Aceptada

ADR-0001. Microservicios por contexto acotado, no por épica

- **Estado:** Aceptada
- **Fecha:** 2026-08-11
- **Alcance:** Toda la plataforma

Contexto

El enunciado exige una arquitectura de microservicios. El backlog está organizado en cinco épicas: usuarios, publicaciones, interacciones sociales, notificaciones y backoffice. Cortar un servicio por épica mezclaría datos de distintos dueños. Por ejemplo, la épica de usuarios incluye la foto de perfil, que es un archivo, y el envío de feedback, que se gestiona desde el backoffice.

Decisión

Cortamos los servicios por contexto acotado, según quién es dueño de cada dato:

- **auth-service:** identidad, credenciales, perfil, preferencias y estado de la cuenta.
- **posts-service:** contenido, interacciones y feed.
- **social-service:** quién sigue a quién.
- **media-service:** archivos.
- **admin-service:** administradores y backoffice.

Un **api-gateway** es la única puerta de entrada (ADR-0007). Cada historia del backlog lleva una etiqueta `service:*` que indica qué servicio es dueño de sus datos.

Alternativas

- Un servicio por épica.
- Un monolito modular, con un solo despliegue.

Consecuencias

A favor:

- Cada servicio tiene un dueño de datos claro y se despliega por separado.
- Las historias de una misma épica se reparten entre servicios sin duplicar tablas.

En contra:

- Hay más llamadas por red: armar el feed requiere a posts, social y auth.
- Una operación que abarca dos servicios ya no puede resolverse con una sola transacción (ADR-0010).

ADR-0002. Una base de datos por servicio, sin claves foráneas entre servicios

- **Estado:** Aceptada
- **Fecha:** 2026-08-19
- **Alcance:** auth, posts, social, media y admin

Contexto

Si dos servicios comparten tablas, un cambio de esquema en uno puede romper al otro, y se pierde la posibilidad de desplegarlos por separado.

Decisión

Cada servicio tiene su propia base PostgreSQL y es el único que la lee y la escribe. Las referencias a datos de otro servicio se guardan como UUID sin clave foránea. Por ejemplo, `posts.author_id` y `follows.follower_id` guardan el id de un usuario de auth. Para leer datos de otro servicio se usa su API (ADR-0010). Cada servicio versiona su esquema con sus propias migraciones: Alembic en Python y goose en Go.

Alternativas

- Una base compartida, con un esquema por servicio.
- Claves foráneas entre tablas de distintos servicios.

Consecuencias

A favor:

- Los esquemas evolucionan de forma independiente.
- La falla de una base no tumba a los demás servicios.

En contra:

- La base no garantiza integridad referencial entre servicios. Si un usuario se da de baja, sus follows quedan en social hasta que exista el evento correspondiente (ADR-0010).
- Las consultas que cruzan datos de varios servicios se resuelven en código, con pedidos en lote.

ADR-0003. PostgreSQL como fuente de verdad y Redis para datos que vencen

- **Estado:** Aceptada
- **Fecha:** 2026-08-13
- **Alcance:** auth, posts y social

Contexto

El enunciado pide al menos dos tipos de base de datos. Además, varios datos tienen una vida corta y conocida: el link de verificación de email dura 24 horas, el de recuperación de contraseña dura 10 minutos, y hay contadores por hora, bloqueos por intentos fallidos y tokens revocados.

Decisión

PostgreSQL guarda todo lo permanente. Redis guarda lo efímero, siempre con tiempo de vida (TTL) y con un prefijo por servicio: `verify:`, `reset:`, `auth:login-attempts:`, `auth:account-lock:user:`, `auth:revoked-token:`, `posts:rate:` y `social:follows:rate:`. Los contadores usan operaciones atómicas de Redis. En producción se usa un Redis gestionado, fuera del cluster.

Alternativas

- Guardar todo en PostgreSQL, con una columna de vencimiento y un proceso periódico de limpieza.

Consecuencias

A favor:

- El vencimiento es automático y no hace falta un proceso de limpieza.
- Los contadores siguen siendo correctos aunque lleguen pedidos simultáneos.

En contra:

- Si Redis pierde sus datos, se pierden los links pendientes y los contadores. Se acepta porque el usuario puede volver a pedirlos.
- Hay un componente más para configurar y operar en cada entorno.

ADR-0004. Python con FastAPI y SQLAlchemy 2.0 sincrónico

- **Estado:** Aceptada
- **Fecha:** 2026-08-11
- **Alcance:** auth, posts, social, media y api-gateway

Contexto

El equipo necesitaba un stack conocido para avanzar rápido, con validación de datos y documentación de la API sin trabajo extra.

Decisión

Usamos FastAPI con Pydantic para validar entradas y salidas, SQLAlchemy 2.0 en modo sincrónico, Alembic para las migraciones, PyJWT, argon2-cffi, y Ruff para lint y formato. Cada servicio sigue las mismas capas, con dependencias en una sola dirección:

- **router:** HTTP, rutas, moldes y códigos de estado.
- **service:** reglas de negocio y transacciones.
- **repository:** el único lugar con SQL.

El gateway usa httpx asíncrono, porque su trabajo es reenviar pedidos y esperar respuestas.

Alternativas

- SQLAlchemy en modo asíncrono.
- Django.

Consecuencias

A favor:

- El código sincrónico es más simple de leer y de testear a esta escala. Es la razón que deja escrita la guía de auth-service.
- La documentación OpenAPI queda disponible en `/docs` sin trabajo extra.

En contra:

- Cada pedido ocupa un hilo mientras espera a la base. Con más carga habría que medir y, si hace falta, pasar a modo asíncrono.

ADR-0005. admin-service escrito en Go

- **Estado:** Aceptada
- **Fecha:** 2026-09-17
- **Alcance:** admin-service

Contexto

El enunciado no permite que todo el backend esté en una sola tecnología. El backoffice es un contexto aislado: sus usuarios son los administradores, no los usuarios de la app, y tiene su propia base.

Decisión

Escribimos admin-service en Go 1.23, con chi como router, pgx para PostgreSQL, goose para las migraciones y golang-jwt. Mantiene las mismas capas que el resto: handler, service y repository. Firma sus propios tokens con un secreto distinto del de la app y hashea las contraseñas con Argon2id, igual que auth.

Alternativas

- Node.js con TypeScript.
- Java con Spring.

Consecuencias

A favor:

- Cumple el requisito de backend políglota en un servicio con poca dependencia de los demás.
- Genera un binario chico que arranca rápido.

En contra:

- El equipo mantiene dos toolchains, dos estilos de test y dos tipos de pipeline.
- La cobertura se mide con otra herramienta. Hoy está en 89,9%, por encima del 85% exigido.

ADR-0006. Sesiones con JWT firmado y revocable

- **Estado:** Aceptada
- **Fecha:** 2026-08-16
- **Alcance:** auth, posts, social y api-gateway

Contexto

Varios servicios necesitan saber quién hace cada pedido sin consultar a auth en cada llamada. A la vez, las historias exigen poder cerrar sesión y que cambiar la contraseña invalide todas las sesiones abiertas.

Decisión

auth emite un JWT firmado con HS256 que dura una hora. Lleva tres datos clave: `sub`, el id del usuario; `token_version`; y `jti`, un id único del token.

- **Cerrar sesión** guarda el `jti` en una lista de revocados en Redis hasta que el token vence.
- **Cambiar o restablecer la contraseña** incrementa `users.token_version`. Todo token con una versión anterior deja de valer, en cualquier dispositivo.

posts y social verifican la firma y el vencimiento por su cuenta, con el secreto compartido. La revocación solo la conoce auth, y el gateway la consulta en cada pedido (ADR-0007).

Alternativas

- Sesiones guardadas en la base, con un identificador opaco.
- Tokens de acceso cortos más tokens de refresco.

Consecuencias

A favor:

- Cada servicio verifica el token localmente, sin costo de red.
- Cerrar sesión y cerrar todas las sesiones funcionan sin guardar las sesiones activas.

En contra:

- El secreto HS256 se comparte entre los servicios Python. Si se filtra en uno, se pueden falsificar tokens para todos. La mejora planificada es firmar con clave privada y verificar con clave pública.
- Sin tokens de refresco, la sesión vence a la hora y hay que volver a iniciar sesión.

ADR-0007. API gateway propio con verificación central del token

- **Estado:** Aceptada
- **Fecha:** 2026-09-14
- **Alcance:** api-gateway y auth

Contexto

La app no debería conocer la dirección de cada servicio. Además, la revocación de tokens (ADR-0006) solo la conoce auth.

Decisión

Un api-gateway escrito en FastAPI recibe todo bajo `/api` y rutea según la primera palabra de la ruta:

Primera palabra	Servicio
auth	auth-service
posts	posts-service
follows , follow-requests , users	social-service
media , files	media-service
admin	admin-service
cualquier otra	responde 404

Para todo lo que no es auth ni admin, el gateway verifica el token con `GET /auth/introspect`, guarda la respuesta 30 segundos en memoria y reenvía el pedido con las cabeceras `X-User-Id` y `X-User-Handle`. auth y admin pasan sin verificar porque manejan su propia autenticación.

Alternativas

- Un gateway administrado, como AWS API Gateway.
- Usar el Ingress del cluster como único punto de entrada, sin gateway propio.

Consecuencias

A favor:

- La app usa una sola URL.
- La revocación de tokens se respeta en todos los servicios.
- La caché evita una llamada a auth en cada pedido.

En contra:

- Un token revocado puede seguir pasando hasta 30 segundos.
- La caché vive en la memoria de un proceso. Con varias réplicas, cada una tendría la suya.
- Mientras tenga una sola réplica, el gateway es un punto único de falla.

ADR-0008. La base garantiza unicidad, idempotencia y contadores

- **Estado:** Aceptada
- **Fecha:** 2026-08-13
- **Alcance:** auth, posts y social

Contexto

Dos pedidos simultáneos pueden pasar un chequeo hecho en código, como "¿ya existe este email?", y terminar escribiendo duplicados. Lo mismo pasa con un me gusta enviado dos veces o con un contador que se lee y se escribe desde el código.

Decisión

Las reglas de unicidad las impone PostgreSQL, no el código:

- **Email único sin importar mayúsculas:** `users.email_normalized` guarda el email en minúsculas y sin espacios, con índice único.
- **Me gusta y guardados idempotentes:** clave primaria compuesta `(post_id, author_id)` e inserción con "si ya existe, no hagas nada".
- **Un retweet por persona:** restricción única `(user_id, post_id)`.
- **Contadores sin carreras:** `like_count`, `reply_count` y `follow_counters` se actualizan con una suma atómica en la base, dentro de la misma transacción que la fila. `retweet_count` se actualiza bloqueando la fila del post con `SELECT ... FOR UPDATE` hasta el commit.

Alternativas

- Validar en código antes de escribir.
- Calcular los contadores con `COUNT(*)` en cada lectura.

Consecuencias

A favor:

- No hay duplicados ni contadores desfasados, aunque lleguen pedidos al mismo tiempo.
- Dar me gusta y guardar son idempotentes, así que la app puede reintentar sin riesgo.

En contra:

- Parte de la lógica vive en el esquema, así que hay que leer los modelos y las migraciones para entenderla completa.

ADR-0009. Paginación por cursor en las listas

- **Estado:** Aceptada
- **Fecha:** 2026-08-31
- **Alcance:** posts y social, y la app

Contexto

Las listas cambian mientras el usuario las recorre. Con paginación por número de página, que usa OFFSET, si entran posts nuevos se repiten elementos al pedir la página siguiente. Además, OFFSET recorre todas las filas que saltea.

Decisión

Las listas se paginan con cursor, también llamado keyset. El cliente recibe un cursor opaco que codifica la fecha y el id del último elemento, y la página siguiente pide lo que está antes de ese punto. Cada consulta tiene un índice que coincide con su orden, por ejemplo `(parent_id, created_at, id)` para las respuestas o `(tag, created_at)` para los hashtags. La app recorre las páginas con infinite queries de TanStack Query.

Alternativas

- Paginación con OFFSET y LIMIT.

Consecuencias

A favor:

- No hay repetidos ni elementos salteados cuando entran datos nuevos.
- El costo de cada página es el mismo, sin importar cuán lejos esté.

En contra:

- No se puede saltar directo a una página por número.
- Cada lista necesita su índice compuesto.

ADR-0010. Comunicación sincrónica entre servicios, en lote y con falla cerrada

- **Estado:** Aceptada
- **Fecha:** 2026-09-10
- **Alcance:** posts, social y auth

Contexto

El feed necesita datos de tres servicios: a quién sigue el usuario, que está en social; los posts, en posts; y los datos públicos y la privacidad de cada autor, en auth. El enunciado además exige al menos una cola asincrónica entre dos servicios.

Decisión

Por ahora los servicios se comunican por HTTP sincrónico, con dos reglas:

- **Pedidos en lote.** Para no hacer un pedido por elemento, posts pide los autores con `POST /auth/users/batch`, de hasta 100 ids, y obtiene las marcas del usuario, es decir me gusta, guardado y retweet, con tres consultas por página.
- **Falla cerrada.** Si auth o social no responden, posts devuelve 503 en lugar de mostrar contenido cuya privacidad no pudo verificar.

La cola queda postergada. La guía de media-service registra la decisión de arrancar con llamadas sincrónicas y migrar al llegar a la unidad de escalabilidad del cronograma. auth ya publica el evento `user_deleted` a través de una interfaz, que hoy solo lo registra en el log.

Alternativas

- Usar una cola de eventos desde el inicio, con SQS o Redis Streams.
- Copiar los datos de los autores en posts.

Consecuencias

A favor:

- Es simple de razonar y depurar, y los datos siempre están actualizados.
- Nunca se muestra contenido de una cuenta protegida por error.

En contra:

- Hay acoplamiento de disponibilidad: si auth cae, el feed cae.
- Sin la cola, la baja de un usuario no se propaga: sus follows quedan en social. Implementarla es trabajo pendiente para la entrega final.

ADR-0011. Baja lógica de usuarios y posts

- **Estado:** Aceptada
- **Fecha:** 2026-08-18
- **Alcance:** auth y posts

Contexto

Borrar filas de verdad rompe los hilos de conversación y deja referencias colgadas en otros servicios, que no tienen clave foránea (ADR-0002).

Decisión

`users.deleted_at` y `posts.deleted_at` marcan la baja en lugar de borrar la fila.

- Un usuario dado de baja no puede iniciar sesión, y el directorio de auth deja de devolverlo. Por eso posts deja de mostrar su contenido.
- Un post borrado conserva su lugar en el hilo con el texto "[Post eliminado]".
- Para dar de baja la cuenta, la app pide reingresar la contraseña.

Alternativas

- Borrado físico, en cascada.

Consecuencias

A favor:

- Se conservan la integridad referencial y las conversaciones.
- Queda registro de qué se borró y cuándo.

En contra:

- Cada consulta tiene que filtrar por `deleted_at`.
- Los datos siguen almacenados después de la baja.

ADR-0012. Despliegue en el cluster EKS compartido de la cátedra

- **Estado:** Aceptada
- **Fecha:** 2026-10-02
- **Alcance:** Los seis servicios del backend

Contexto

La cátedra provee un cluster EKS compartido llamado `tds-cluster`, un namespace por grupo con una cuota de 8 pods, un balanceador compartido y CloudFront. La base de datos la tiene que conseguir cada grupo.

Decisión

- **Un Deployment por servicio**, con una réplica, estrategia Recreate y chequeos de readiness y liveness sobre `/health`.
- **Migraciones al arrancar:** el contenedor corre `alembic upgrade head` antes de levantar uvicorn.
- **Entrada única:** un Ingress manda `/api` al gateway.
- **Pipeline de CD en GitHub Actions:** asume un rol de AWS por OIDC, sin claves guardadas; construye la imagen; la sube a ECR etiquetada con el commit; arma el Secret con GitHub Secrets; aplica los manifiestos y espera el rollout.
- **Fuera del cluster:** PostgreSQL y Redis.

Alternativas

- Varias réplicas con RollingUpdate.
- PostgreSQL y Redis dentro del cluster.

Consecuencias

A favor:

- Usa seis pods de ocho, así que queda lugar para un servicio más.
- No hay credenciales de AWS de larga duración.

En contra:

- Recreate deja el servicio sin responder unos segundos en cada despliegue.
- Con una réplica no hay alta disponibilidad.
- El pipeline de CD no vuelve a correr los tests: depende de que la CI haya pasado en el pull request.

ADR-0013. Testing con dependencias inyectadas, base real y mínimo de cobertura

- **Estado:** Aceptada
- **Fecha:** 2026-09-04
- **Alcance:** Todos los backends

Contexto

La cátedra exige 85% de cobertura unitaria en cada servicio y tests de integración para todas las integraciones del backend, ejecutados en la CI. Los servicios dependen de PostgreSQL, Redis, un servidor de mail y otros servicios.

Decisión

Las dependencias externas se inyectan con `Depends` de FastAPI: `get_db`, `get_redis` y `get_email_sender`. En los tests se reemplazan así:

- **PostgreSQL:** una base real y dedicada, con una transacción que se deshace al final de cada test.
- **Redis:** fakeredis, un Redis en memoria.
- **Mail:** un EmailSender que no envía nada.
- **Otros servicios:** el cliente HTTP también se inyecta y se reemplaza por uno falso. El gateway simula los servicios con respX.

Los mocks se atan a la interfaz real con `spec`, para que fallen si la interfaz cambia. La CI corre dentro de Docker: Ruff, una prueba de las migraciones que las aplica y las deshace, y los tests, con un mínimo de 85% que hace fallar el pipeline. Las credenciales salen de GitHub Secrets.

Alternativas

- SQLite en memoria para los tests.
- Mocks sin `spec`.

Consecuencias

A favor:

- Los tests de integración encuentran errores de SQL y de migraciones que SQLite no mostraría.
- Los backends tienen entre 89,9% y 99,3% de cobertura.

En contra:

- Los tests necesitan un PostgreSQL disponible.
- La app mobile no tiene mínimo de cobertura en su CI y está en 61%. Llevarla a 85% es trabajo pendiente.

ADR-0014. App mobile con Expo, TanStack Query y Zustand

- **Estado:** Aceptada
- **Fecha:** 2026-08-25
- **Alcance:** mobile-app

Contexto

La app principal tiene que ser mobile, y el equipo necesitaba probarla en dispositivos reales sin experiencia previa en desarrollo nativo para iOS y Android.

Decisión

Usamos React Native 0.81 con Expo SDK 54 y TypeScript.

- **Estado del servidor** (feed, posts, perfil): TanStack Query, con claves de caché y actualizaciones optimistas que se corrigen con la respuesta del servidor.
- **Estado propio de la app:** Zustand. La sesión guarda el token cifrado en SecureStore; el tema se guarda en AsyncStorage.
- **Formularios, navegación e idiomas:** React Hook Form con Zod, React Navigation e i18next.

Cuando cambia el token, la caché se vacía entera para no mostrar datos de otra sesión.

Alternativas

- Flutter.
- Redux para todo el estado.

Consecuencias

A favor:

- Una sola base de código para iOS y Android, que se prueba con Expo Go.
- La interfaz responde al instante gracias a las actualizaciones optimistas.

En contra:

- Las notificaciones push con Firebase de la época E.4 requieren un build propio: Expo Go no alcanza.
- La versión de Expo Go del teléfono tiene que coincidir con el SDK del proyecto.